

Praktikum 3 — Exploratory Data Analysis and Visualisation

Data Science • Magister Ilmu Komputer, FMIPA UGM Companion to **Chapter 3**. Assignment weight: **3%**.

What you will do

1. Summarise a variable properly — robustly, not just `.mean()`.
2. Choose histogram bins by rule instead of by accident.
3. Fix skew with a transformation and verify it worked.
4. Tell linear from monotone association (Pearson vs Spearman).
5. Reproduce Anscombe's quartet and Simpson's paradox.
6. Build figures that obey the encoding hierarchy.

EDA generates hypotheses; it does not test them. If you explore fifty variables and then test the strongest correlation you found, its p-value is meaningless. Keep that separation in mind all the way through.

```
import sys
from pathlib import Path

ROOT = Path.cwd()
while not (ROOT / "checks.py").exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent
sys.path.insert(0, str(ROOT))

import checks
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt

DATA = ROOT / "data"
FIGS = ROOT / "output" / "figures"
FIGS.mkdir(parents=True, exist_ok=True)

# seaborn is optional throughout this course; matplotlib does everything we need
try:
    import seaborn as sns
    sns.set_theme(style="whitegrid", palette="colorblind")
    HAS_SNS = True
except ImportError:
    HAS_SNS = False

plt.rcParams.update({
    "figure.dpi": 110, "savefig.dpi": 200, "savefig.bbox": "tight",
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.grid": True, "grid.alpha": 0.3,
    "font.size": 10, "axes.titleweight": "bold",
})
print("seaborn available:", HAS_SNS)
```

```
sales = pd.read_csv(DATA / "sales_clean.csv", parse_dates=["order_date"])
print(sales.shape)
sales.head()
```

1. Describing a variable honestly (DEMO)

`.describe()` gives you the mean and standard deviation. On skewed data both are misleading, and neither tells you the shape. This is the summary the course uses instead.

```
def describe_full(s):
    s = pd.Series(s).dropna()
    return pd.Series({
        "n": len(s),
        "mean": s.mean(),
        "median": s.median(),
        "trimmed_10": stats.trim_mean(s, 0.10),
        "std": s.std(ddof=1),
        "iqr": s.quantile(0.75) - s.quantile(0.25),
        "mad": (s - s.median()).abs().median(),
        "cv": s.std(ddof=1) / s.mean(),
        "skew": s.skew(),
        "excess_kurt": s.kurtosis(),
        "p01": s.quantile(0.01),
        "p99": s.quantile(0.99),
    })

amount = sales["amount"]
print(describe_full(amount).round(3).to_string())

print(f"mean / median ratio : {amount.mean() / amount.median():.3f}")
print(f"share below the mean: {(amount < amount.mean()).mean():.1%}")
print()
print("If well over half the data lies below the mean, reporting the mean as")
print("'the typical order' describes almost nobody.")
```

TASK 3.1

Report the **skewness** of `amount`. Is it symmetric or strongly skewed?

```
kemencengan = None      # <- the skewness of sales["amount"]

checks.task("3.1", kemencengan)
```

2. Binning is a decision (DEMO)

A histogram's only parameter is the bin width, and it changes what you see. Sturges assumes normality and under-bins large or skewed samples; **Freedman–Diaconis** uses the robust IQR and adapts to the data.

$$k_{\text{Sturges}} = \lceil \log_2 n \rceil + 1 \quad h_{\text{FD}} = 2 \cdot \frac{\text{IQR}}{n^{1/3}}$$

```
n = len(amount)
iqr = amount.quantile(0.75) - amount.quantile(0.25)
k_sturges = int(np.ceil(np.log2(n)) + 1)
h_fd = 2 * iqr / n ** (1 / 3)
k_fd = int(np.ceil((amount.max() - amount.min()) / h_fd))

print(f"n = {n:,}")
print(f"Sturges          : {k_sturges} bins")
print(f"Freedman-Diaconis: width {h_fd:,.0f} -> {k_fd} bins")
```

```
fig, axes = plt.subplots(1, 3, figsize=(13, 3.4))
for ax, bins, title in zip(
    axes, [8, "fd", 300],
    ["Too few (8 bins)", f"Freedman-Diaconis ({k_fd} bins)", "Too many (300)"]):
    ax.hist(amount, bins=bins, color="#2E6F9E", edgecolor="white", linewidth=0.3)
    ax.set(title=title, xlabel="order amount (Rp)")
axes[0].set_ylabel("orders")
fig.suptitle("The same data, three bin widths", y=1.04, fontweight="bold")
fig.savefig(FIGS / "03-binning.png")
plt.show()
```

Never accept the first histogram. Redraw at two or three widths. A feature that survives every reasonable binning is real; one that appears at a single width is not.

The ECDF has no tuning parameter at all, which makes it the honest fallback:

```
fig, ax = plt.subplots(figsize=(5.5, 3.4))
x = np.sort(amount)
ax.plot(x, np.arange(1, len(x) + 1) / len(x), color="#173F5F")
ax.set(xlabel="order amount (Rp)", ylabel="cumulative proportion",
       title="Empirical CDF – no bandwidth to choose")
for q in (0.25, 0.5, 0.75):
    ax.axhline(q, color="grey", lw=0.6, ls=":")
    ax.text(x.max() * 0.98, q + 0.01, f"Q{int(q*4)}", ha="right", fontsize=8,
           color="grey")
plt.show()
```

3. Transformations (DEMO + TASK)

Right-skewed positive variables are the norm in economic data, and most classical methods behave better on a transformed scale.

```
logged = np.log(amount)
bc, lam = stats.boxcox(amount)

print(f"skew, raw      : {stats.skew(amount):7.3f}")
print(f"skew, log      : {stats.skew(logged):7.3f}")
print(f"skew, Box-Cox   : {stats.skew(bc):7.3f}    (lambda = {lam:.4f})")
print()
print("Box-Cox picked lambda close to 0, which *is* the log transform -")
print("the method independently rediscovers it.")
```

```
fig, axes = plt.subplots(1, 2, figsize=(10, 3.4))
axes[0].hist(amount, bins="fd", color="#9C3535", edgecolor="white", linewidth=0.3)
axes[0].set(title=f"Raw (skew {stats.skew(amount):.2f})", xlabel="amount")
axes[1].hist(logged, bins="fd", color="#356B3F", edgecolor="white", linewidth=0.3)
axes[1].set(title=f"log(amount) (skew {stats.skew(logged):.2f})",
            xlabel="log amount")
plt.show()
```

Transforming changes what a model *means*. A linear model on `log(y)` estimates **multiplicative** effects: a coefficient of 0.08 means "about 8% higher", not "0.08 higher". And back-transforming a predicted mean gives the geometric mean, which sits below the arithmetic one.

TASK 3.2

Report the skewness **after** the log transform.

```
kemencengan_log = None

checks.task("3.2", kemencengan_log)
```

4. Pearson, Spearman, and why the gap matters (DEMO)

- **Pearson** measures *linear* association, and is sensitive to outliers.
- **Spearman** is Pearson on ranks: it detects any *monotone* relationship and is robust, because ranks compress extremes.

A large gap between them is a reliable signal that you should transform before fitting anything linear.

```
rng = np.random.default_rng(1)
x = rng.uniform(1, 10, 200)
y = np.exp(x / 2) * rng.lognormal(0, 0.25, 200)    # monotone, strongly convex

print(f"Pearson r          = {stats.pearsonr(x, y)[0]:.3f}")
print(f"Spearman rho       = {stats.spearmanr(x, y)[0]:.3f}")
print(f"Pearson on log(y) = {stats.pearsonr(x, np.log(y))[0]:.3f}")

fig, axes = plt.subplots(1, 2, figsize=(10, 3.4))
axes[0].scatter(x, y, s=10, alpha=0.6, color="#2E6F9E")
axes[0].set(title="Raw: curved, so Pearson understates", xlabel="x", ylabel="y")
axes[1].scatter(x, np.log(y), s=10, alpha=0.6, color="#356B3F")
axes[1].set(title="log(y): linear, so Pearson agrees", xlabel="x",
            ylabel="log y")
plt.show()
```

TASK 3.3

You have a strictly increasing but strongly curved relationship. Which coefficient will be closer to 1 — `"pearson"` or `"spearman"` ?

```
koefisien = None          # "pearson" or "spearman"

checks.task("3.3", koefisien)
```

5. Anscombe's quartet (DEMO)

Four datasets with, to two decimal places, identical means, variances, correlations and fitted regression lines — and radically different structure. The lesson is operational, not cute: **always plot the data**.

```
x123 = [10, 8, 13, 9, 11, 14, 6, 4, 12, 7, 5]
x4 = [8, 8, 8, 8, 8, 8, 8, 19, 8, 8, 8]
y1 = [8.04, 6.95, 7.58, 8.81, 8.33, 9.96, 7.24, 4.26, 10.84, 4.82, 5.68]
y2 = [9.14, 8.14, 8.74, 8.77, 9.26, 8.10, 6.13, 3.10, 9.13, 7.26, 4.74]
y3 = [7.46, 6.77, 12.74, 7.11, 7.81, 8.84, 6.08, 5.39, 8.15, 6.42, 5.73]
y4 = [6.58, 5.76, 7.71, 8.84, 8.47, 7.04, 5.25, 12.50, 5.56, 7.91, 6.89]

ans = pd.concat([
    pd.DataFrame({"dataset": "I", "x": x123, "y": y1}),
    pd.DataFrame({"dataset": "II", "x": x123, "y": y2}),
    pd.DataFrame({"dataset": "III", "x": x123, "y": y3}),
    pd.DataFrame({"dataset": "IV", "x": x4, "y": y4}),
], ignore_index=True)

rows = []
for name, g in ans.groupby("dataset"):
    slope, intercept = np.polyfit(g.x, g.y, 1)
    rows.append({"dataset": name,
                 "mean_x": g.x.mean(), "mean_y": g.y.mean(),
                 "var_x": g.x.var(ddof=1), "var_y": g.y.var(ddof=1),
                 "corr": g.x.corr(g.y),
                 "slope": slope, "intercept": intercept})
summary = pd.DataFrame(rows).set_index("dataset").round(3)
print(summary.to_string())
```

```
fig, axes = plt.subplots(2, 2, figsize=(9, 6.4))
notes = {"I": "the case a linear model is for",
        "II": "exactly quadratic – residuals are a parabola",
        "III": "one outlier drags the slope",
        "IV": "one high-leverage point creates the whole fit"}
for ax, (name, g) in zip(axes.ravel(), ans.groupby("dataset")):
    ax.scatter(g.x, g.y, s=28, color="#2E6F9E", zorder=3)
    xs = np.linspace(2, 20, 50)
    b, a = np.polyfit(g.x, g.y, 1)
    ax.plot(xs, a + b * xs, color="#9C3535", lw=1.2)
    ax.set(title=f"{name}: {notes[name]}", xlim=(2, 20), ylim=(2, 14))
    ax.title.set_fontsize(9)
fig.suptitle("Anscombe's quartet – identical statistics, different data",
            y=1.02, fontweight="bold")
fig.savefig(FIGS / "03-anscombe.png")
plt.show()
```

6. Simpson's paradox (DEMO)

An association present in every subgroup can **reverse** in the aggregate. This is the sharpest form of the confounding problem.

```

adm = pd.DataFrame({
    "dept": ["A", "A", "B", "B"],
    "gender": ["M", "F", "M", "F"],
    "applied": [800, 100, 150, 900],
    "admitted": [512, 70, 30, 162],
})
adm["rate"] = adm["admitted"] / adm["applied"]
print("Within each department:")
print(adm.to_string(index=False))

agg = adm.groupby("gender")[["applied", "admitted"]].sum()
agg["rate"] = agg["admitted"] / agg["applied"]
print("\nAggregated over departments:")
print(agg.round(4).to_string())

```

Within department A women are admitted at a *higher* rate (70% vs 64%); in B the rates are near-equal. Yet in aggregate men are at 57% and women at 23%.

The reversal is entirely explained by the confounder: department B is far more selective, and women applied overwhelmingly to it.

Neither table is "the truth." Which one answers your question depends on the *causal* question you are asking — and data alone cannot decide that.

```

# The same structure in the sales data, without the social content.
# Conditioning on channel changes the picture of region performance.
overall = (sales.groupby("region", observed=True)["amount"]
           .mean().sort_values(ascending=False))
print("Mean order amount by region (marginal):")
print(overall.round(0).to_string())

print("\nMean order amount by region, conditioned on channel:")
print(sales.pivot_table(index="region", columns="channel",
                        values="amount", aggfunc="mean",
                        observed=True).round(0).to_string())

print("\nChannel mix differs by region – which is what makes the marginal")
print("comparison hard to interpret:")
print((sales.pivot_table(index="region", columns="channel",
                        values="amount", aggfunc="size", observed=True)
       .pipe(lambda d: d.div(d.sum(axis=1), axis=0))
       .round(3).to_string()))

```

7. Reading a correlation matrix (DEMO)

Look for three things: strong correlation with the **target** (candidate predictors), strong correlation **among predictors** (multicollinearity, which destabilises linear coefficients), and any correlation of exactly 1.0 between columns that should be distinct — a duplicate, or a leaked target.

```

num = sales.select_dtypes("number")
corr = num.corr(method="spearman")

pairs = (corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
         .stack().sort_values(key=abs, ascending=False))
print("Strongest pairs (Spearman):")
print(pairs.head(8).round(3).to_string())

fig, ax = plt.subplots(figsize=(5.2, 4.2))
im = ax.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)
ax.set_xticks(range(len(corr)), corr.columns, rotation=45, ha="right")
ax.set_yticks(range(len(corr)), corr.columns)
for i in range(len(corr)):
    for j in range(len(corr)):
        ax.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center", va="center",
                fontsize=7,
                color="white" if abs(corr.iloc[i, j]) > 0.6 else "black")
ax.set_title("Spearman correlation")
ax.grid(False)
fig.colorbar(im, ax=ax, shrink=0.8)
plt.show()

```

`amount` correlates near-perfectly with `quantity` and `unit_price` — because it is their product. That is a constructed identity, not a discovery, and feeding all three to a linear model would be a multicollinearity problem.

8. Encoding: making the right comparison easy (DEMO)

Cleveland and McGill ranked visual channels by decoding accuracy:

Rank	Channel	Use for
1	Position on a common scale	anything; the default
3	Length	magnitudes from a zero baseline
4	Angle, slope	trends — why pie charts are weak
5	Area	rough magnitude only
7	Colour hue	nominal categories, ≤ 7 of them

Encode the most important comparison in position.

```

by_region = (sales.groupby("region", observed=True)["amount"]
              .agg(["mean", "count"]).sort_values("mean"))

fig, axes = plt.subplots(1, 2, figsize=(11, 3.6))

# BAD: unsorted, truncated axis, decorative colour
bad = sales.groupby("region", observed=True)["amount"].mean()
axes[0].bar(range(len(bad)), bad.values,
            color=plt.cm.rainbow(np.linspace(0, 1, len(bad))))
axes[0].set_xticks(range(len(bad)), bad.index, rotation=30, ha="right")
axes[0].set_ylim(bad.min() * 0.985, bad.max() * 1.005) # truncated!
axes[0].set_title("Misleading: truncated axis, rainbow, unsorted")

# GOOD: sorted, zero baseline, one colour, direct labels
axes[1].barh(range(len(by_region)), by_region["mean"], color="#2E6F9E")
axes[1].set_yticks(range(len(by_region)), by_region.index)
axes[1].set_xlim(0, by_region["mean"].max() * 1.18)
for i, (v, c) in enumerate(zip(by_region["mean"], by_region["count"])):
    axes[1].text(v * 1.02, i, f"{v/1e6:.2f} jt (n={c:,})", va="center",
                 fontsize=8)
axes[1].set_title("Honest: sorted, zero baseline, direct labels")
axes[1].set_xlabel("mean order amount (Rp)")
axes[1].grid(axis="y", visible=False)
fig.savefig(FIGS / "03-encoding.png")
plt.show()

```

The left panel makes a 1.5% difference look like a fourfold one. Both panels show the same numbers.

9. Overplotting (DEMO)

A scatter of thousands of points is a solid rectangle that shows only the extent of the data. Fix it deliberately, and say in the caption which fix you used.

```

fig, axes = plt.subplots(1, 3, figsize=(13, 3.4))
q, p = sales["quantity"], sales["unit_price"]

axes[0].scatter(q, p, s=12)
axes[0].set(title="Overplotted", xlabel="quantity", ylabel="unit price")

axes[1].scatter(q + np.random.default_rng(0).normal(0, 0.12, len(q)), p,
                s=8, alpha=0.06, color="#173F5F")
axes[1].set(title="Jitter + alpha=0.06", xlabel="quantity")

hb = axes[2].hexbin(q, p, gridsize=28, cmap="Blues", mincnt=1)
axes[2].set(title="Hexagonal binning", xlabel="quantity")
fig.colorbar(hb, ax=axes[2], shrink=0.8, label="count")
plt.show()

```

10. Take-home (assessed)

A. Anscombe by hand. Verify that the four datasets share means, variances and correlations to two decimals (done above), then write one sentence per panel describing precisely what a linear model gets *wrong*.

B. Build your own paradox. Construct a synthetic dataset in which a treatment appears beneficial within each of two subgroups but harmful overall. State the confounder explicitly, and say which of the two conclusions answers the question *"should an individual take the treatment?"*

C. Criticise a figure. A colleague proposes: *"a 3-D exploded pie chart of market share for eleven competitors, coloured with the rainbow colormap, with a legend on the right."* Name **four** distinct defects and the principle each violates, then produce a corrected figure using the sales data.

D. An EDA report. Write `eda_report(df, target)` returning: the profile table, `describe_full` for every numeric column, the ten strongest Spearman correlations with `target`, and a **warning** for any column whose correlation with the target exceeds 0.98. Explain in a comment why that last check matters.

Next week: we stop describing and start deciding whether what we see is real.